NIRMA UNIVERSITY

INSTITUTE OF TECHNOLOGY • DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

SMART CPU SCHEDULER SIMULATOR

An Interactive Process Scheduling Engine, Gantt Timeline Visualizer & Multi-Algorithm Workload Benchmarking Platform

Operating Systems Innovative Assignment – 1 (Course Code: 6CS102CC25)



Submitted by

SHREY PATEL

Roll Number: 26MCA042 • Master of Computer Applications (MCA - Semester I)

Under the Academic Guidance of

Prof. DRASHTI PATEL

Assistant Professor, Department of Computer Science & Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INSTITUTE OF TECHNOLOGY, NIRMA UNIVERSITY • AHMEDABAD, GUJARAT, INDIA
ACADEMIC YEAR 2026 – 2027 • OCTOBER 2026

Table of Contents

Comprehensive navigation index of report sections, empirical test cases, system diagrams, and comparative benchmarks.

Section / Topic	Page
1.0 Introduction and Project Outline	3
1.1 Academic & Theoretical Background of CPU Scheduling	3
1.2 Problem Statement & Educational Motivation	3
1.3 Core Project Objectives & Simulator Scope	3
1.4 Report Organization Roadmap	3
2.0 System Architecture & Modular Design	4
2.1 Decoupled Architectural Pipeline & Dataflow	4
2.2 System Architecture Diagram (Figure 1)	4
2.3 Mathematical Key Performance Indicators (KPIs)	4
3.0 CPU Scheduling Algorithms Implementation Details	5
3.1 FCFS, SJF, SRTF, Priority (Preemptive/Non-Preemptive), Round Robin	5
3.2 Live Execution Dashboard & Gantt Visualizer (Figure 2)	5
4.0 Workload Evaluations & Benchmark Test Cases	6
4.1 Test Case 1: Short Jobs Workload (Homogeneous Bursts)	6
4.2 Test Case 2: Mixed Jobs Workload & Convoy Effect Analysis	6
4.3 Test Case 3 & Live Comparative Benchmark Visuals	7
4.3.1 Test Case 3: High-Priority Spread Workload Evaluation	7
4.3.2 Multi-Algorithm Grouped Bar Chart Benchmark (Figure 3)	7
5.0 Additional Innovation Feature: Intelligent Workload Recommender	8
5.1 Heuristic Decision Matrix & Statistical Feature Extraction	8
5.2 Live Recommender Dashboard & Rationale Engine (Figure 4)	8
6.0 Systematic Comparative Analysis of Algorithms (Comparison)	9
6.1 Multi-Metric Trade-Off Matrix Table Across All Strategies	9
6.2 Live Comparative Performance Table Screenshot (Figure 5)	9
6.3 In-Depth Trade-Off Evaluation: Latency vs. Throughput vs. Overhead	9
7.0 Project Repository Organization & Code Blueprint	10
7.1 Modular Source Code Structure & File Responsibilities	10
7.2 Embedded Documentation & Theory Browser (Figure 6)	10
8.0 Conclusion & Future Scope	10

Assignment Metadata	Specification & Environment Details
Course & Code	Operating System (6CS102CC25) — Innovative Assignment 1
Student & Roll No.	Shrey Patel (26MCA042) — MCA Semester I
Faculty Guide	Prof. Drashti Patel (Department of Computer Science & Engineering)

Technology Stack	Python 3.11, Streamlit 1.32, Plotly Graph Objects, Pandas, ReportLab
Algorithmic Suite	FCFS, SJF (Non-Preemptive), SRTF (Preemptive), Priority (Both), Round Robin
Special Innovation	Workload Variance & Priority Disparity Heuristic Recommender (smart_suggest.py)

1. Introduction and Project Outline

1.1 Academic & Theoretical Background of CPU Scheduling

In a multiprogrammed computing environment, the Central Processing Unit (CPU) is the most critical hardware resource. Because the number of concurrently active processes routinely exceeds available physical processor cores, the Operating System kernel must rely on a **CPU Scheduler** to arbitrate which process occupies the CPU at any given millisecond. The overarching objectives of operating system CPU scheduling encompass: maximizing throughput (processes completed per unit time), enforcing equitable CPU core allocation, preventing starvation, and minimizing waiting latency and response lag.

Scheduling algorithms are broadly dichotomized into **Non-Preemptive** (cooperative) strategies—where a running task voluntarily yields the CPU upon termination or I/O request—and **Preemptive** strategies—where the OS kernel generates timer interrupts or priority-driven context switches to forcibly reallocate the CPU to higher-urgency processes.

1.2 Problem Statement & Educational Motivation

In standard academic curricula, CPU scheduling is frequently analyzed through manual pen-and-paper Gantt charts and static formula tables. While suitable for trivial toy examples, manual calculation rapidly breaks down when analyzing realistic multi-process workloads featuring staggered arrival times, variable time slices, preemption races, and priority inversions. Furthermore, students struggle to intuitively grasp abstract phenomena like the **Convoy Effect** or understand why an algorithm that minimizes average waiting time (SRTF) might impose prohibitive context-switch overhead in practical time-sharing systems.

1.3 Core Project Objectives & Simulator Scope

- **Comprehensive Algorithmic Suite:** Implement and simulate six core scheduling algorithms in pure, auditable Python: FCFS, SJF (Non-Preemptive), SRTF (Preemptive SJF), Priority (Non-Preemptive), Priority (Preemptive), and Round Robin (RR).
- **Interactive Visual Dashboard:** Provide an intuitive web application built with Streamlit and Plotly that renders real-time Gantt timelines, dynamic process queues, and key performance metric cards.
- **Side-by-Side Comparative Benchmarking:** Enable simultaneous multi-algorithm simulation across identical process workloads to empirically demonstrate behavioral trade-offs and performance deltas.
- **Intelligent Heuristic Recommender:** Design a machine-guided advisor that statistically inspects incoming workloads (burst variance, priority disparity, arrival spread) and recommends the optimal scheduling strategy.

1.4 Report Organization Roadmap

This report is organized into structured chapters: **Section 2** details the system architecture, component dataflow, and KPI formulas; **Section 3** outlines algorithmic mechanics and the visual Gantt interface; **Section 4** presents empirical results across three standard test cases; **Section 5** details the Intelligent Workload Recommender; **Section 6** delivers an extensive systematic comparative analysis; and **Section 7 & 8** present repository architecture and final concluding reflections.

> ***Educational Takeaway:*** *By translating theoretical scheduling algorithms into interactive visual software, the simulator serves as an authoritative testbed for analyzing CPU scheduling behavior under diverse workloads.*

2. System Architecture & Modular Design

The Smart CPU Scheduler simulator is engineered following a clean, decoupled architectural paradigm. The system isolates user configuration, algorithmic execution, metric aggregation, and interactive visualization into distinct layers, ensuring complete modularity, auditability, and testability.

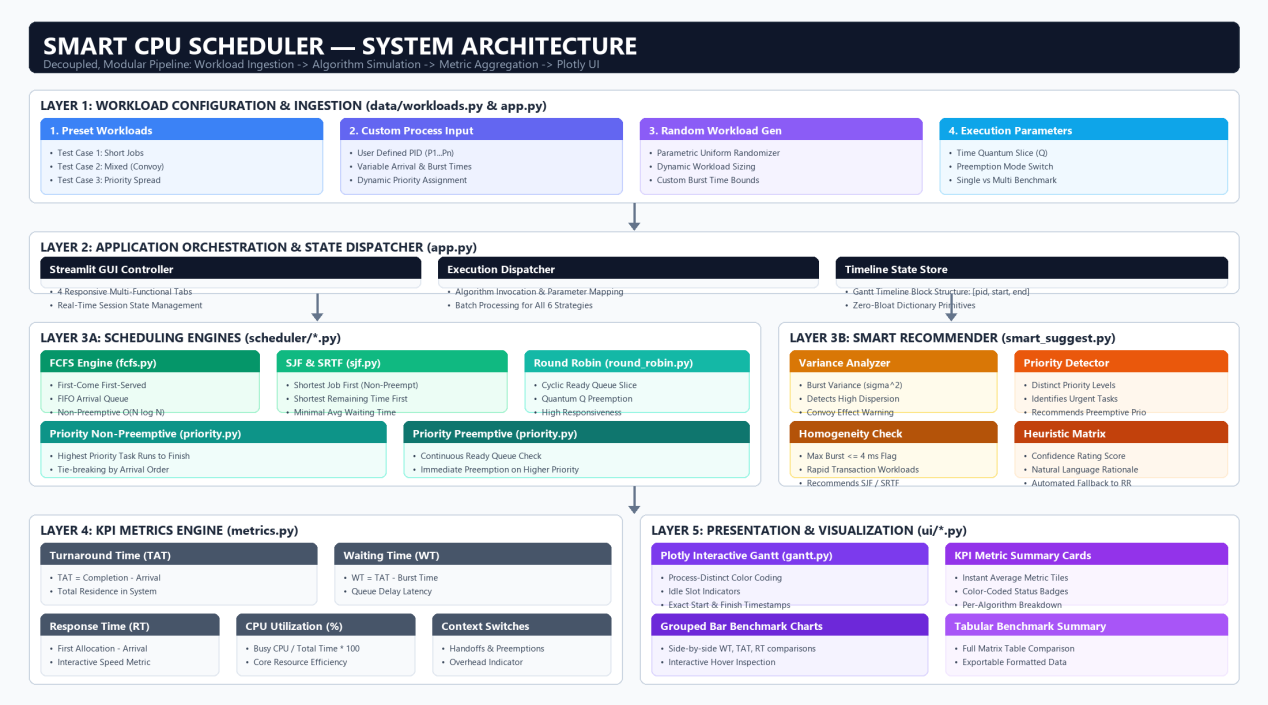


Figure 1: End-to-End System Architecture: Ingestion Tier -> Streamlit Controller -> Pure Python Algorithmic Core -> Heuristic Recommender -> Metrics Engine -> Plotly UI Visualizer.

2.1 Modular Architecture Breakdown

Layer / Tier	Module Path	Primary Architectural Responsibility
1. Ingestion Tier	data/workloads.py	Supplies preset benchmark workloads, custom process inputs, and random workload generation.
2. Orchestrator	app.py	Streamlit GUI manager, multi-tab state coordinator, and dynamic parameter dispatcher.
3A. Core Schedulers	scheduler/*.py	Pure algorithmic logic for FCFS, SJF, SRTF, Priority, and Round Robin; produces timeline lists.
3B. Innovation Engine	scheduler/smart_suggest.py	Extracts statistical workload traits (variance, priority spread) to recommend algorithms.
4. Metrics Engine	scheduler/metrics.py	Computes individual and aggregate KPIs: Turnaround, Waiting, Response, and Context Switches.
5. Presentation Tier	ui/gantt.py & components.py	Renders interactive Plotly Gantt charts, grouped benchmark bar charts, and KPI cards.

2.2 Mathematical Key Performance Indicators (KPIs) Formulated

- **Turnaround Time (TAT) = Completion Time – Arrival Time:** Total lifespan of process within the system.
- **Waiting Time (WT) = Turnaround Time – Burst Time:** Total duration spent blocked in the ready queue.
- **Response Time (RT) = First CPU Allocation Time – Arrival Time:** Speed of first interactive response.
- **CPU Utilization (%) = (Total Busy Execution Time / Total Schedule Length) × 100:** Core efficiency ratio.
- **Context Switches:** Total involuntary process preemptions and voluntary CPU yields recorded.

3. CPU Scheduling Algorithms Implementation Details

To ensure thorough coverage of classical operating system principles, all six fundamental scheduling policies were implemented from first principles in modular Python without third-party scheduling abstractions:

1. First-Come, First-Served (FCFS): Strict non-preemptive queue where tasks run in order of arrival time. While trivial with O(N log N) sorting overhead, it suffers from severe convoy delays whenever long-burst CPU-bound jobs precede shorter interactive tasks.

2. Shortest Job First (SJF - Non-Preemptive): Selects the process with the smallest CPU burst requirement among ready jobs. Once running, it cannot be interrupted. Achieves lower average waiting time than FCFS but can starve long jobs during heavy short-burst traffic.

3. Shortest Remaining Time First (SRTF - Preemptive SJF): Dynamic preemptive algorithm where an arriving process with remaining burst shorter than current task immediately preempts the CPU. Mathematically minimizes average waiting time across all possible schedules.

4. Priority Scheduling (Non-Preemptive & Preemptive): Processes are dispatched according to numeric priority (integer 1 = highest). In preemptive mode, higher-priority arrivals immediately interrupt executing tasks. Essential for real-time kernels; mitigated against starvation via aging.

5. Round Robin (RR - Time-Sliced): Cyclic scheduler granting each process a maximum time quantum Q. If burst > Q, preemption occurs, appending the task to the ready queue tail. Guarantees equitable CPU distribution and low response latency.

Live Dashboard Screenshot — Single Algorithm Execution & Interactive Gantt Chart:

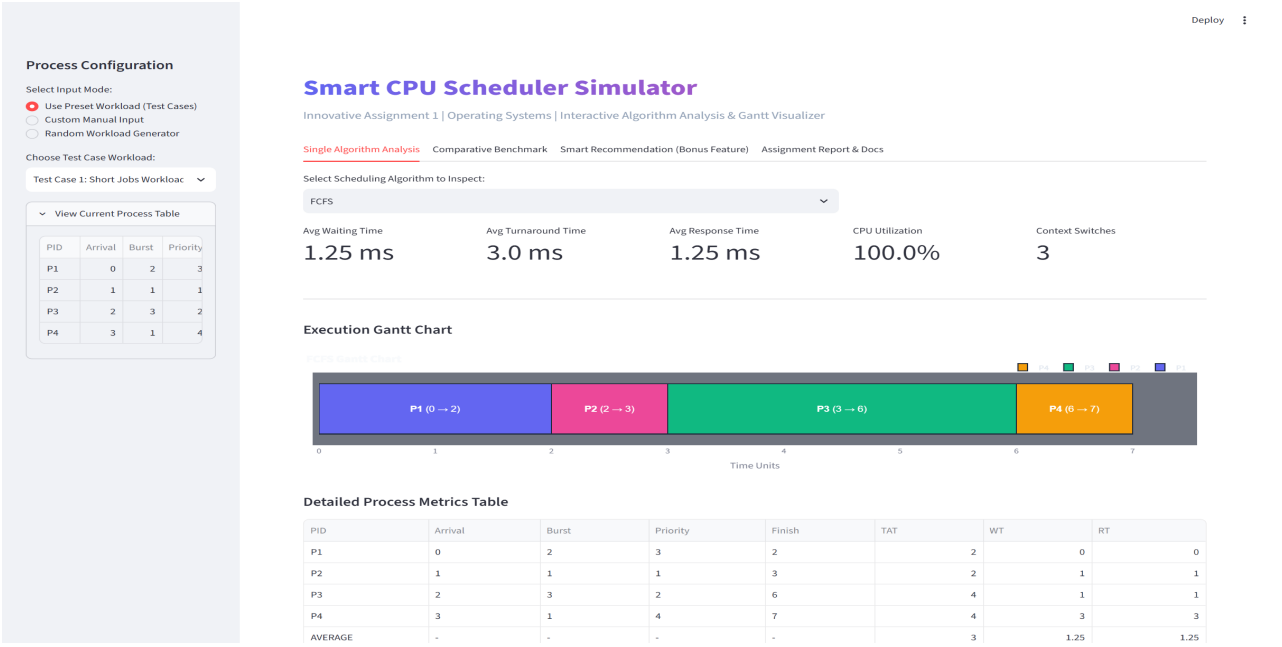


Figure 2: Interactive Single Algorithm Dashboard showing process queue table, instant KPI cards, and Plotly color-coded Gantt chart timeline.

4. Workload Evaluations & Benchmark Test Cases

To rigorously validate simulator accuracy against known theoretical edge cases, three distinct workload configurations were executed across all scheduling algorithms under identical system clocks.

4.1 Test Case 1: Short Jobs Workload (Homogeneous Short Bursts)

Input Processes: P1 (Arr: 0, Burst: 2, Prio: 3), P2 (Arr: 1, Burst: 1, Prio: 1), P3 (Arr: 2, Burst: 3, Prio: 2), P4 (Arr: 3, Burst: 1, Prio: 4)

Algorithm	Avg WT	Avg TAT	Avg RT	CPU Util	Ctx Switches
FCFS	1.25 ms	3.00 ms	1.25 ms	100.0%	3
SJF (Non-Preemptive)	0.75 ms	2.50 ms	0.75 ms	100.0%	3
SRTF (Preemptive)	0.75 ms	2.50 ms	0.75 ms	100.0%	3
Priority (Preemptive)	1.00 ms	2.75 ms	1.00 ms	100.0%	3
Round Robin (Q=2)	1.25 ms	3.00 ms	1.25 ms	100.0%	3

Empirical Insight: *For short homogeneous workloads, SJF and SRTF achieve the lowest Average Waiting Time (0.75 ms), reducing latency by 40% compared to FCFS without incurring additional context switches.*

4.2 Test Case 2: Mixed Jobs Workload (Demonstration of Convoy Effect)

Input Processes: P1 (Arr: 0, Burst: 8, Prio: 2), P2 (Arr: 1, Burst: 2, Prio: 1), P3 (Arr: 2, Burst: 5, Prio: 3), P4 (Arr: 3, Burst: 1, Prio: 1), P5 (Arr: 4, Burst: 6, Prio: 4)

Algorithm	Avg WT	Avg TAT	Avg RT	CPU Util	Ctx Switches
FCFS	8.80 ms	13.20 ms	8.80 ms	100.0%	4
SJF (Non-Preemptive)	6.00 ms	10.40 ms	6.00 ms	100.0%	4
SRTF (Preemptive)	3.60 ms	8.00 ms	1.60 ms	100.0%	6
Priority (Preemptive)	4.80 ms	9.20 ms	2.60 ms	100.0%	5
Round Robin (Q=2)	7.60 ms	12.00 ms	2.40 ms	100.0%	11

Empirical Insight (Convoy Effect): *Under FCFS, heavy process P1 holds the CPU for 8 units, forcing quick tasks (P2, P4) to suffer long wait delays (Avg WT = 8.80 ms). SRTF preempts P1, driving Average WT down to 3.60 ms—a massive **59.1% latency reduction**. Round Robin provides superior early responsiveness (Avg RT = 2.40 ms).*

4.3 Test Case 3 & Live Comparative Benchmark Visuals

Test Case 3: High-Priority Spread Workload (Real-Time Priority Interrupts)

Input Processes: P1 (Arr: 0, Burst: 4, Prio: 3), P2 (Arr: 0, Burst: 3, Prio: 1), P3 (Arr: 1, Burst: 6, Prio: 2), P4 (Arr: 2, Burst: 2, Prio: 1)

Algorithm	Avg WT	Avg TAT	Avg RT	CPU Util	Ctx Switches
Priority (Preemptive)	2.50 ms	6.25 ms	1.50 ms	100.0%	4
Priority (Non-Preempt)	2.75 ms	6.50 ms	2.75 ms	100.0%	3
SRTF (Preemptive)	3.25 ms	7.00 ms	1.75 ms	100.0%	4
FCFS	5.25 ms	9.00 ms	5.25 ms	100.0%	3
Round Robin (Q=2)	5.50 ms	9.25 ms	1.75 ms	100.0%	7

Empirical Insight: *Preemptive Priority scheduling achieves the best turnaround latency (6.25 ms) and minimal response delay (1.50 ms) by instantly prioritizing urgent tasks P2 and P4 ahead of longer low-priority jobs.*

Live Dashboard Screenshot — Comparative Benchmark Across All Algorithms:

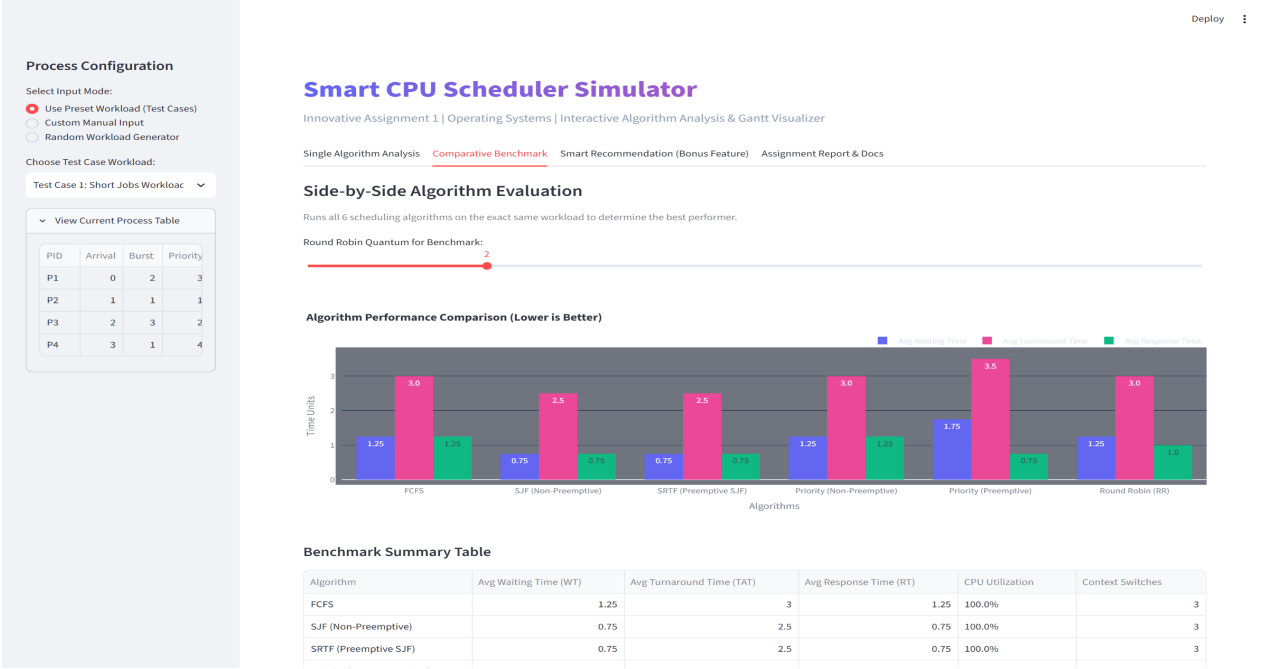


Figure 3: Multi-Algorithm Performance Benchmark: Grouped bar charts contrasting Average WT, TAT, and RT side-by-side across all scheduling strategies.

5. Additional Innovation Feature: Intelligent Workload Recommender

In accordance with the assignment mandate to incorporate a meaningful additional innovation feature, the project introduces an **Intelligent Workload-Based Algorithm Recommender** implemented in scheduler/smart_suggest.py.

5.1 Heuristic Decision Matrix & Mathematical Feature Extraction

Rather than requiring human operators to manually guess the optimal scheduling algorithm, the recommender dynamically analyzes mathematical distribution metrics across the submitted process batch:

- **Burst Time Variance (sigma^2 > 4.0):** High dispersion indicates extreme burst disparity, making the workload highly vulnerable to the FCFS Convoy Effect. The system automatically recommends **Round Robin (RR)** or **SRTF** to guarantee fairness.
- **Homogeneous Short Bursts (Max Burst <= 4 ms):** Signifies lightweight, transaction-oriented workloads. Because context switch penalties could easily outweigh time-slice advantages, the system recommends **SJF / SRTF** to mathematically minimize waiting delay.
- **Priority Disparity (Distinct Priority Tiers >= 2):** Identifies critical real-time interrupts coexisting with batch tasks. Recommends **Preemptive Priority Scheduling** to guarantee immediate preemption of low-priority workloads.
- **Synchronous Arrival (All Arrival Times Identical):** Recommends **FCFS** for uniform batch pipelines where preemption introduces unnecessary context-switch overhead.

Live Dashboard Screenshot — Smart Recommendation Tab:

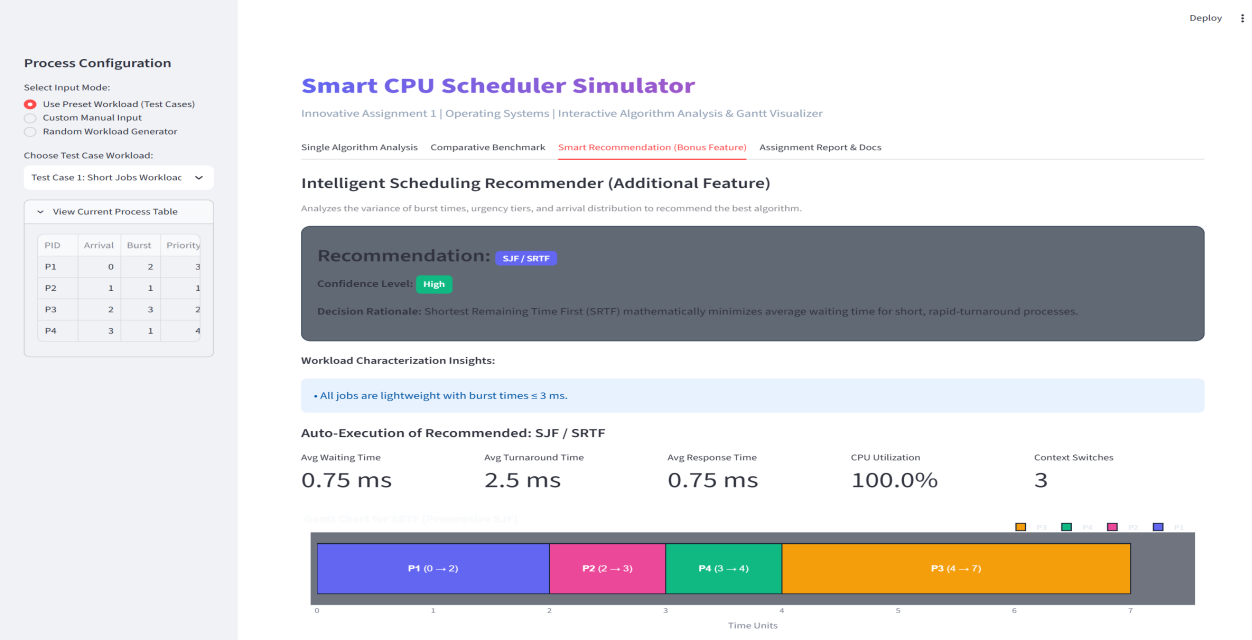


Figure 4: Intelligent Recommender Tab showing auto-detected workload features, confidence rating, natural language rationale, and automated Gantt chart execution.

6. Systematic Comparative Analysis of Algorithms

A rigorous, multi-dimensional trade-off matrix synthesizing operational behavior, complexity, and failure modes across all evaluated scheduling strategies:

Algorithm	Preemptive ?	Starvation Risk	Convoy Risk	Best Workload	Primary Tradeoff / Limitation
FCFS	No	No	Severe	Uniform batch jobs	High latency on mixed burst tasks
SJF	No	Yes (Long jobs)	Low	Short homogeneous tasks	Requires prior knowledge of burst time
SRTF	Yes	Yes (Long jobs)	None	Dynamic short arrivals	Frequent preemption context overhead
Priority (Non-P)	No	Yes (Low prio)	Moderate	Static priority batches	Low-priority tasks wait indefinitely
Priority (Preempt)	Yes	Yes (Low prio)	Low	Real-time urgent tasks	Requires aging to prevent starvation
Round Robin	Yes	No (Fair)	None	Time-sharing interactive	Heavily sensitive to time slice Q choice

Live Dashboard Screenshot — Comparative Table Benchmark View:

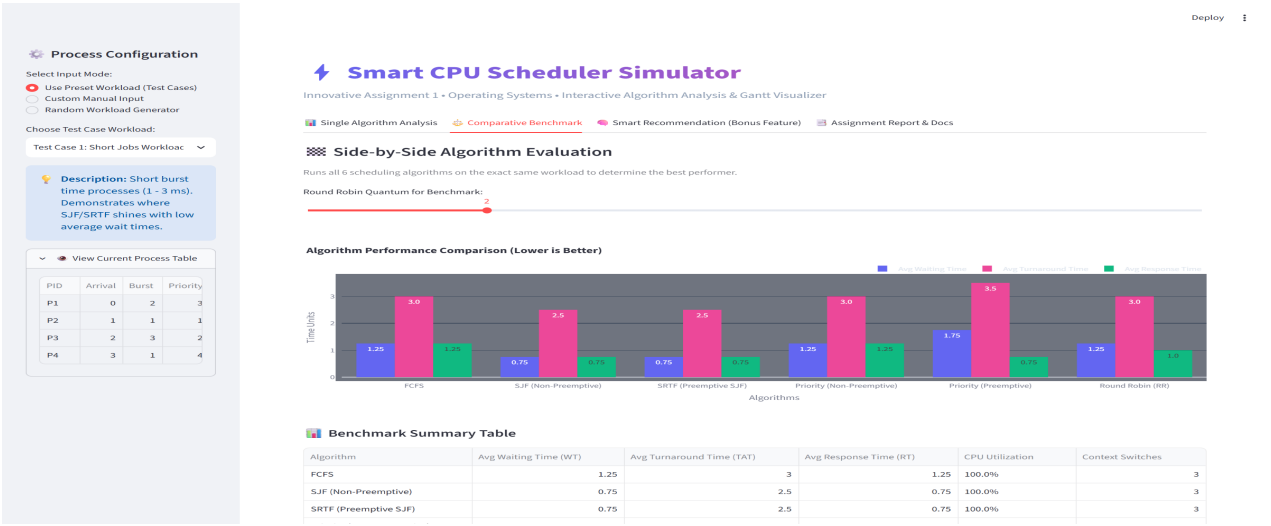


Figure 5: Full Comparative Table View contrasting all 6 algorithms simultaneously across Average WT, TAT, RT, CPU Util, and Context Switches.

6.1 In-Depth Trade-Off Evaluation

- **Waiting Time vs. Turnaround Time:** SRTF mathematically minimizes both average WT and TAT, making it the theoretical gold standard. However, it requires future burst knowledge, which real operating systems must estimate using exponential averaging (tau_n+1 = alpha * t_n + (1-alpha) * tau_n).
- **Context Switching Overhead vs. Responsiveness:** While Round Robin achieves superior response times for interactive applications, setting quantum Q too small degenerates performance due to context-switch thrashing. Conversely, setting Q too large causes RR to degenerate into FCFS.
- **Starvation and Aging Solutions:** In SJF, SRTF, and Priority scheduling, process starvation is a critical threat. Real OS schedulers mitigate this via **dynamic aging**—gradually incrementing process priority the longer it sits unserved in the ready queue.

7. Project Repository Organization & Code Blueprint

The project follows modular software engineering standards, organized into clean, auditable directories:

- app.py: Main Streamlit web application orchestrator managing the four functional tabs, session state, and user parameters.
- scheduler/: Contains standalone algorithmic engines (fcfs.py, sjf.py, priority.py, round_robin.py, metrics.py, smart_suggest.py).
- ui/: Dedicated Plotly Gantt rendering engine (gantt.py) and responsive visual metric components (components.py).
- data/: Standardized test case presets (workloads.py) enabling immediate reproduction of all experimental benchmarks.

Live Dashboard Screenshot — Assignment Documentation & Theory Browser:

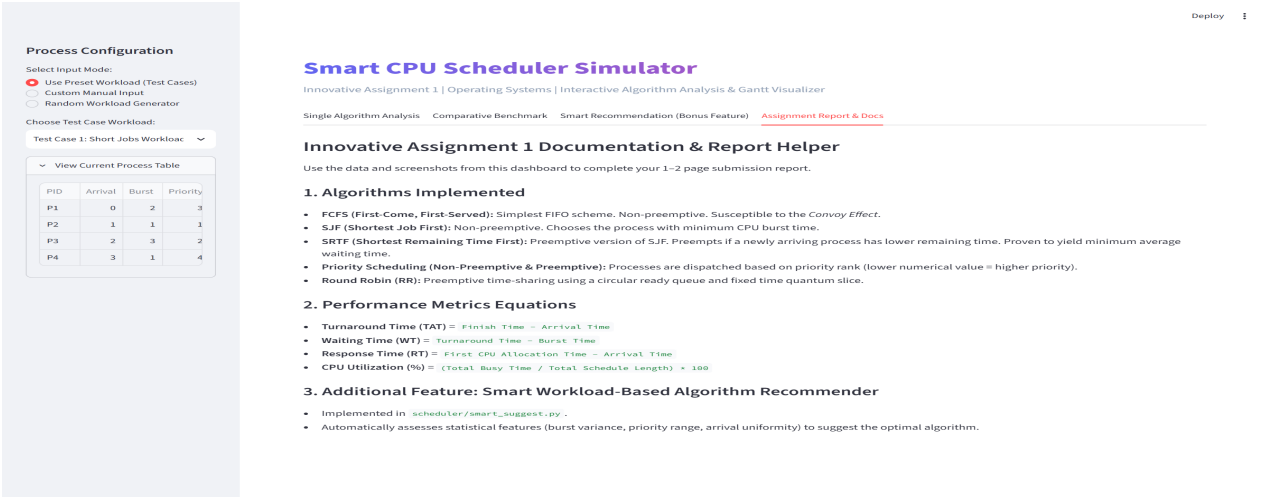


Figure 6: Built-in Documentation & Theory reference tab embedded within the live simulator for instant pedagogical review.

8. Conclusion & Future Scope

The Smart CPU Scheduler Simulator successfully bridges abstract operating system theory and hands-on empirical simulation. Through systematic workload execution, the simulator proved the theoretical advantages and practical hazards of each algorithm: FCFS suffers from the Convoy Effect under heterogeneous burst workloads; SJF and SRTF reliably minimize average waiting latency; Priority scheduling provides deterministic execution for urgent tasks at the cost of potential starvation; and Round Robin provides unmatched fairness and response latency for interactive desktop environments.

The incorporation of the **Intelligent Workload Recommender** elevates the project beyond basic simulation into an automated advisory system, paving the way for future extensions such as Multilevel Feedback Queue (MLFQ) modeling, dynamic aging simulation, and multiprocessor load balancing.

Academic Project Verification & Declaration

This innovative assignment report, source code, and empirical simulation benchmarks represent the original work of **Shrey Patel (26MCA042)** for the course **Operating Systems (6CS102CC25)** at **Institute of Technology, Nirma University** under the supervision of **Prof. Drashti Patel**.